

Arquitectura de Software (Microservicios): Hub Edge AI Agrícola

1. Explicación de la Arquitectura de Software

El software del Hub Central se ha diseñado utilizando un enfoque basado en **Microservicios Contenedorizados**. Esta estrategia arquitectónica abandona los sistemas monolíticos tradicionales en favor de componentes modulares, independientes y débilmente acoplados. El objetivo principal es garantizar que, si un servicio falla (por ejemplo, el módulo de captura de video), el resto del sistema (como la ingesta crítica de telemetría y el riego) continúe operando sin interrupción, cumpliendo así con los estrictos estándares del sector agroindustrial [cite: 1, 10].

Todo el entorno se orquesta utilizando **Podman** (licencia Apache 2.0), una tecnología daemonless que aísla cada proceso, mejora la seguridad y asegura que el sistema pueda distribuirse comercialmente como un paquete propietario cerrado y libre de regalías.

La arquitectura se estratifica en cuatro capas lógicas (Gateway de Ingesta, Persistencia Multimodal, Core API Lógico y Presentación), conectadas internamente mediante redes virtuales de contenedores. Además, el diseño sigue el patrón de *Clean Architecture*, preparando módulos "Stubs" (como los de *Function Calling* y la base de datos vectorial *pgvector*) para que la futura integración de Inteligencia Artificial (SLM) local se realice como un microservicio satélite (plug-and-play) sin necesidad de reescribir el código base.

2. Desglose de las Cuatro Capas de Microservicios

2.1 Capa de Comunicación e Ingesta (Edge Gateway)

El embudo de entrada del sistema. Aísla la volatilidad del mundo real (señales de radio,

fluctuaciones de red) de la base de datos.

- **Contenedor 1 - Eclipse Mosquitto (Broker MQTT):** Funciona como un servicio de publicación/suscripción. Recibe las tramas JSON que el nodo Gateway LoRa inyecta vía USB.
- **Contenedor 2 - Worker Ingesta (Python):** Demonio de fondo (Background Worker). Se suscribe a los tópicos de Mosquitto, limpia el ruido eléctrico de las lecturas, normaliza los datos y realiza escrituras por lotes (batch) hacia la base de datos para no saturarla.
- **Contenedor 3 - Worker Multimedia (Bash/Python):** Capturador RTSP que extrae periódicamente fotogramas clave (keyframes) de las cámaras IP y los escribe directamente en el sistema de archivos local.

2.2 Capa de Persistencia Multimodal (Data & GIS)

La "caja fuerte" del sistema. Procesa Big Data y datos geográficos, todo respaldado por PostgreSQL y sus extensiones para evitar multiplicidad de gestores.

- **Contenedor 4 - PostgreSQL (Motor Principal):** Base de datos relacional. Configurada obligatoriamente con Write-Ahead Logging (WAL) para prevenir corrupción ante apagones súbitos.
- **Componente Nativo - Big Data (Series Temporales):** Usa particionamiento declarativo de PostgreSQL para organizar los millones de registros de telemetría (humedad, CE, etc.) por mes, garantizando velocidad de consulta a lo largo de los años.
- **Extensión Aislada - PostGIS (Motor GIS):** Almacena geometrías (latitud/longitud, polígonos). Su comunicación vía TCP/IP blindo el código comercial contra la licencia GPLv2.
- **Extensión AI - pgvector:** Almacén de embeddings preparado para indexar contextos semánticos (RAG) para futuras IAs.

2.3 Capa de Lógica de Negocio y Core API

El centro de mando desarrollado en Python. Orquesta reglas, seguridad e interactúa con el frontend o IAs.

- **Contenedor 5 - FastAPI Central (Core):** Microservicio asíncrono que expone la información y procesa comandos.
 - **Motor Determinista:** Reglas If-Then-Else clásicas para control automático de riego (ej. "Activar relé si Humedad < 30%"). Envía órdenes a Mosquitto

para que lleguen al campo.

- **RBAC (Role-Based Access Control):** Sistema de tokens JWT. Diferencia quién solicita un dato o comando (Operario vs. Agrónomo) impidiendo ejecuciones no autorizadas.
- **Geo-REST API:** Traduce la telemetría y los mapas de PostGIS a formato GeoJSON para consumo directo por ARCGIS o mapas web.
- **AI Stubs (Function Calling):** Funciones envolventes de los actuadores listas para ser invocadas vía API por un futuro modelo de IA (SLM).

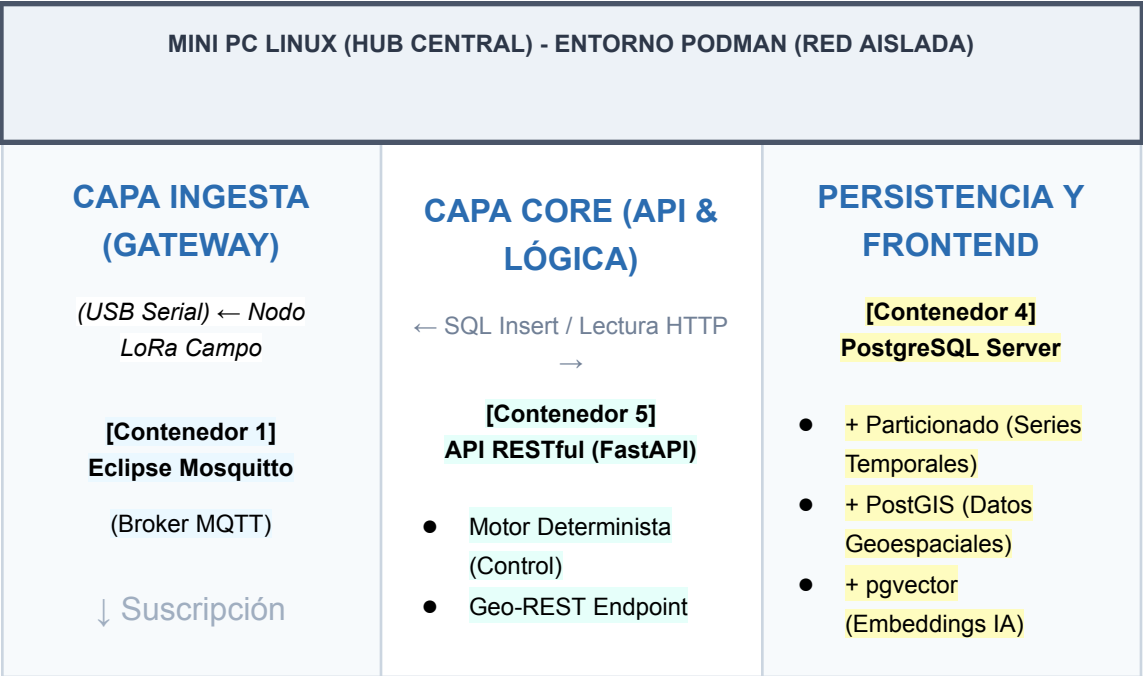
2.4 Capa de Presentación (Frontend)

El punto de contacto humano, accesible offline desde la intranet de la finca.

- **Contenedor 6 - NGINX (Servidor Web):** Sirve los archivos estáticos. Actúa como servidor para la Progressive Web App (PWA) desarrollada en React/Vue y provee las imágenes fotográficas guardadas por el Worker Multimedia.

3. Diagrama Completo de Funcionamiento de Microservicios

El siguiente diagrama detalla la arquitectura de contenedores (Podman) dentro del Hub (Mini PC Linux), el flujo de datos y la orquestación lógica.



MINI PC LINUX (HUB CENTRAL) - ENTORNO PODMAN (RED AISLADA)

(Tópicos) ↓

[Contenedor 2]
Python Worker Ingesta

(Limpia y Parsea Datos)

[Contenedor 3]
Python/Bash RTSP
Worker

(Captura imágenes de
Cámaras IP)

(GeoJSON)

- Seguridad JWT
(RBAC Roles)
- AI Stubs (Function
Calling)
← SQL Query (CRUD) →

↑ Proxy Reverso / HTTP ↓

[Contenedor 6]
NGINX (Web Server)

- Sirve UI Frontend
(React/Vue)
- Sirve Fotos/Archivos
Estáticos

*Salida → Navegador
Local / App*